BU EC528 · Demo 3

Load Balancing Lab

From a round-robin prototype to a lab for testing routing, load, and failures.

Daniel Thomas · Kimone Walker · Neev Jain · Sean O’Connor

Setup → Lab → Compare

Project background

Traffic can overload one server while others have room. Testing a new routing policy in production can affect real users.

Repeatable traffic

Replay the same arrivals and service times

Routing and capacity

Change the policy, traffic rate, or server setup

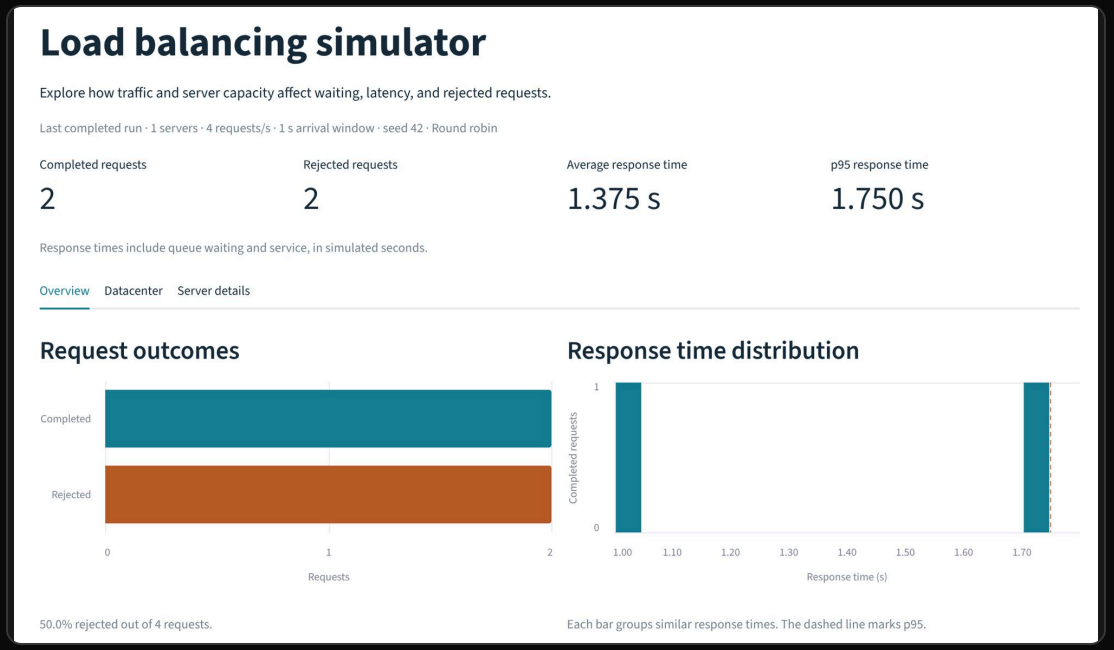
Measured outcomes

Compare queueing, rejections, and response times

From prototype to experiment lab

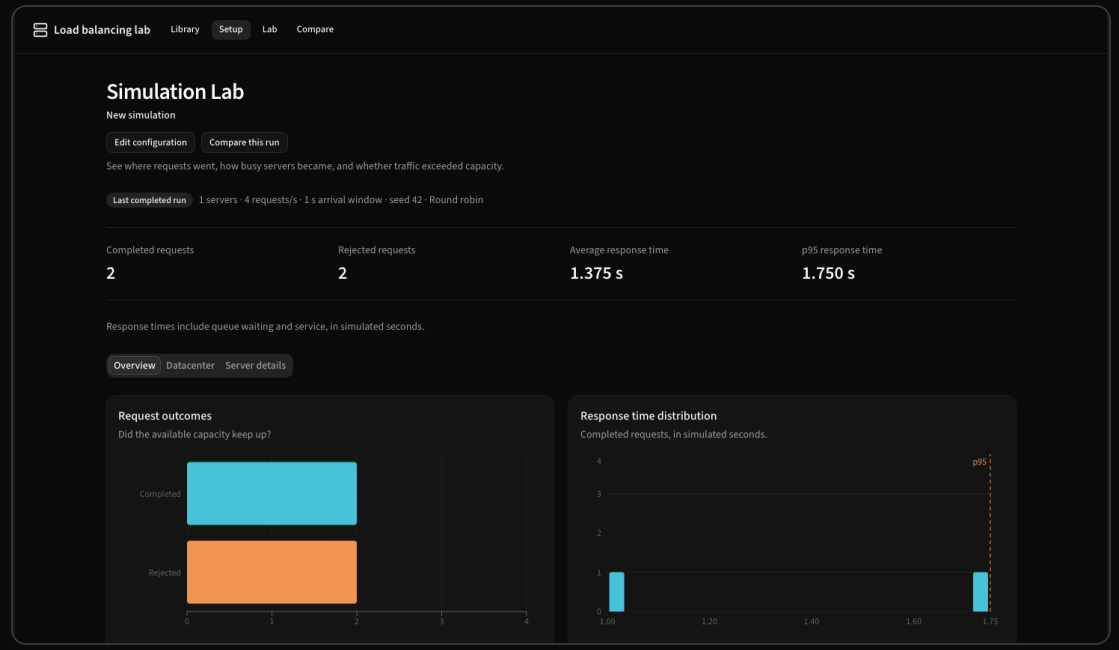
Demo 2

Streamlit · round robin



Demo 3

Next.js Lab · Python API



Same overload example · 2 completed · 2 rejected · 1.75 s p95

What changed since Demo 2

Routing reads server load

Added least-active requests and compared it with round robin on identical traffic.

Experiments can be replayed

Export the configuration and seed, then replay and verify the result.

The engine handles larger runs

Removed arrival-list shifts and bounded the data sent to the browser.

More realistic workloads

Added CPU and I/O stages, live regional traffic, and a seeded shard outage.

Demo 2 commitments · Compare policies · Save and replay · Measure a speedup

Why we changed the frontend

Next.js Lab

Edit inputs, compare runs side by side, and scrub playback

Python API

Validate settings and return summaries or playback pages

Simulation engine

Owns the clock, routing policies, queues, and metrics

We wanted a richer experiment interface. The API lets the browser use the same Python engine as the benchmark scripts.

What we can simulate now

Single pool	CPU → I/O wait → CPU again
Multiple regions	Client → Route by RTT → Near region
Sharded service	Coordinator → Shard A + B → Join

Pools and regions support live traffic changes. Sharded runs also support serial calls, deadlines, and seeded outages.

Counting queued work cut tail latency

Round robin

Rejected **0.00%**

P95 latency **2.13 s**

Least active requests

Rejected **2.91%**

P95 latency **5.24 s**

Least outstanding

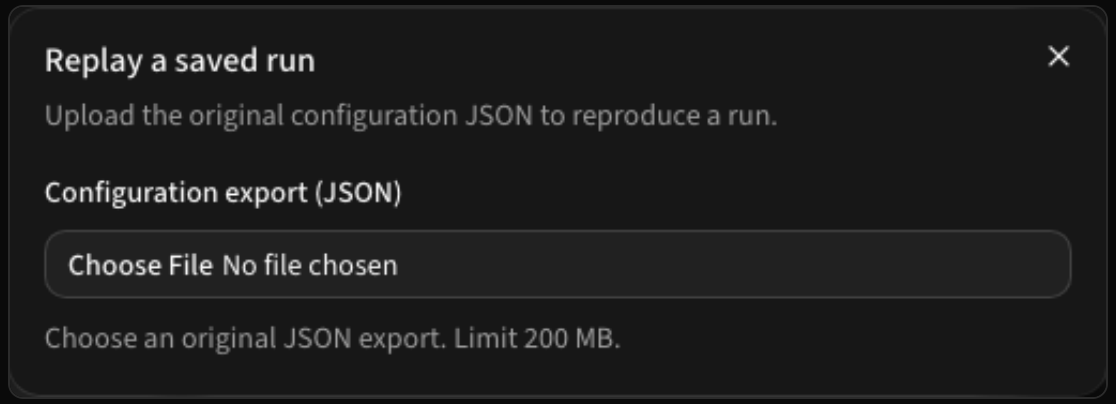
Rejected **0.00%**

P95 latency **1.23 s**

Same traffic at 90% of nominal capacity. Least outstanding counts queued work and had the lowest p95.

A saved run can be replayed exactly

1. Export configuration and seed
2. Replay from the saved JSON
3. Match the full result



Both saved examples reproduced their full summaries.

How we benchmarked

Hold inputs fixed

Round robin · seed 42 · 10k and 100k requests

Profile, then time

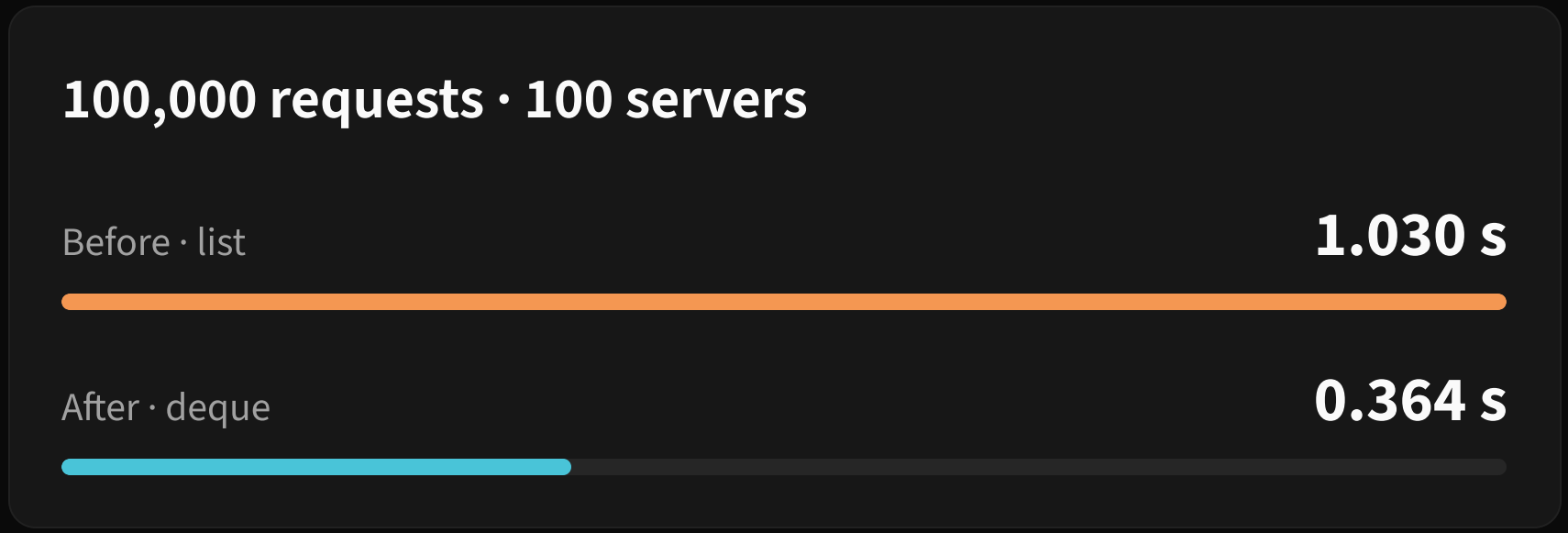
Find hotspots. Take the median of 3 fresh, unprofiled runs per case.

Check the outputs

Compare full-summary hashes, including events and metrics.

Same machine, Python version, and configurations before and after. Fleets of 10, 100, and 1,000 servers.

The same simulation ran 2.83× faster



Profiling led us to arrivals being removed from the front of a list. Switching to deque.popleft preserved every result. At 10,000 requests, the same change took 38.9 ms down to 34.3 ms.

Playback stopped sending the full trace

7.87 MB before

The initial response carried every event and response time.

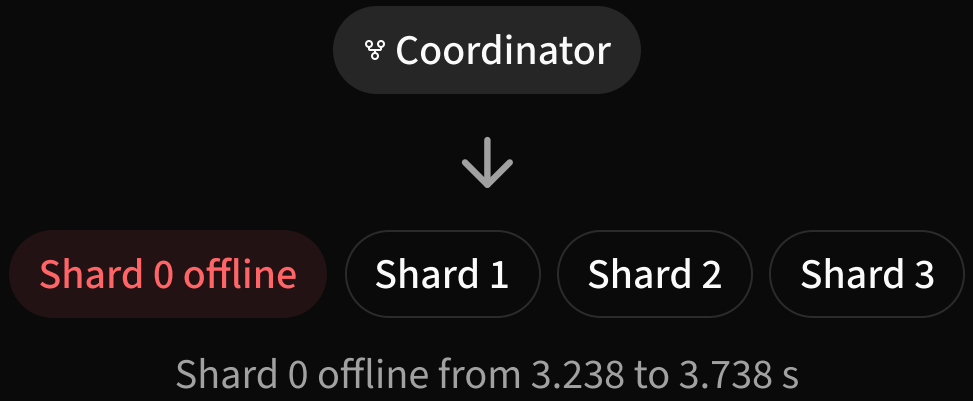
{ } 4.10 kB after

The initial response carries metrics, a histogram, and a run ID.

Scrub playback → Fetch a snapshot or event page

Same 100,000-request, 100-server case. Every paginated event matched the original trace.

One outage, two outcomes



Outage failures out of 200 requests

One required reply **7**

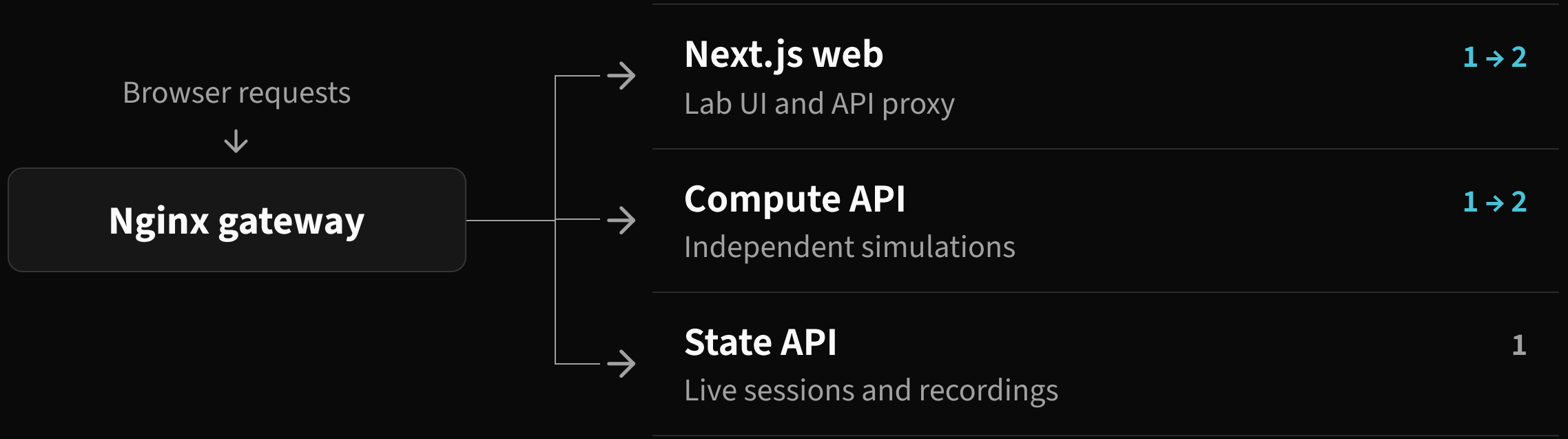
Four required replies **25**

With every shard online, all 200 finished in both cases.

Advanced experiment · Four required replies increase exposure to one failed shard.

Deploy with Docker or Kubernetes

docker Compose | kubernetes



Existing sessions and recordings stayed reachable while web and compute scaled.

Evidence and reproducibility

Routing comparison	10 servers, 90% nominal load, seeds 42–44. P95 is the mean of three per-run percentiles over completed requests.
Engine benchmark	Raw timings and hashes in experiments/results/d3-06 and d3-07. Payload sizes and page checks in experiments/results/a-11.
Shard outage preview	Seed 42, 50 requests/s for 4 s, 200 ms deadline, 4 ms shard round trip. One simulated outage, fixed total work per parent.



Deployment and scaling commands are in `docs/docker-compose.md` and `docs/kubernetes.md`.